

Universidad Nacional Autónoma de México

Facultad de Ingeniería



Sistemas Operativos

Investigación:

Therac - 25

Alumna:

Perez Baez Jessica Estefani

Fecha:

9/11/2026

1. Introducción

Para esta exposición escogí el caso del Therac-25, un acelerador lineal para radioterapia fabricado por Atomic Energy of Canada Limited (AECL). Entre junio de 1985 y enero de 1987 este equipo dio sobredosis de radiación a seis pacientes, y tres de ellos murieron por las lesiones (Leveson y Turner, 1993). Lo que más me llamó la atención es que las causas que se lograron identificar no fueron fallas mecánicas extrañas. Fueron errores que estudiamos en esta materia, una condición de carrera entre dos tareas que compartían variables y un contador de 8 bits que se desbordaba.

No quiero quedarme en la historia del caso. Mi objetivo es explicar cómo ocurrieron esos dos errores a nivel de código, reproducirlos con programas pequeños en C usando hilos POSIX y mostrar cómo se evitan con los mecanismos de sincronización que vimos en clase. Al final reviso cómo se manejan hoy los errores internos en equipos que dependen por completo del software, y por qué eso no contradice lo que enseña este caso.

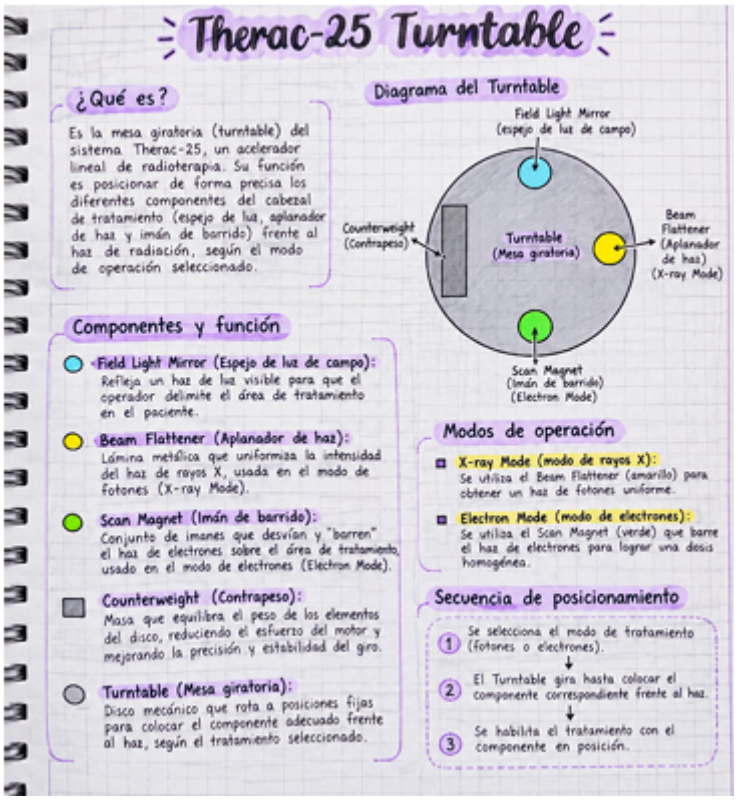
Una aclaración importante es que el código original del Therac-25 nunca se publicó. Los programas de este trabajo son simulaciones que escribí a partir de la descripción técnica de Leveson y Turner (1993), con los mismos nombres de rutinas y variables para que se pueda seguir la relación. Todos los compilé con gcc en Linux y las salidas que aparecen son las que obtuve al ejecutarlos.

2. Cómo funcionaba el Therac-25

2.1 Dos modos de tratamiento y una mesa giratoria

El Therac-25 podía tratar al paciente de dos formas. En el modo de electrones usaba directamente un haz de electrones de entre 5 y 25 MeV. En el modo de rayos X lanzaba electrones de 25 MeV contra un blanco metálico, y los rayos X que salían de ese choque eran los que llegaban al paciente. Como el blanco absorbe gran parte de la energía, el modo de rayos X necesita una corriente de electrones mucho mayor que el modo de electrones (Leveson y Turner, 1993).

Para cambiar de modo, una mesa giratoria (turntable) colocaba distintos accesorios en la trayectoria del haz. En la posición de electrones ponía unos imanes de barrido que reparten el haz sobre el área a tratar. En la de rayos X ponía el blanco y un filtro aplanador. Había una tercera posición, la de luz de campo, que no debe usarse para irradiar porque solo tiene un espejo y una luz para acomodar al paciente. Tres microinterruptores le decían a la computadora en cuál de las tres posiciones estaba la mesa.



De aquí sale la condición más peligrosa de toda la máquina, y conviene tenerla en mente para las siguientes secciones. Si el acelerador produce la corriente alta del modo de rayos X pero la mesa no tiene el blanco enfrente, el paciente recibe directamente un haz de electrones concentrado. Eso fue justo lo que pasó en los accidentes de Tyler y de Yakima.

2.2 ¿El problema fue depender del software?

Al investigar encontré que muchas fuentes resumen el caso como "el error fue confiar en el software". Creo que eso no es del todo correcto. Es cierto que el Therac-25 controlaba por software más cosas que sus antecesores, el Therac-6 y el Therac-20, y que AECL quitó varios interbloqueos de hardware que esos modelos sí tenían (Leveson y Turner, 1993). Pero los aceleradores actuales están controlados completamente por computadora y no por eso son peligrosos. El problema fue quitar una protección sin poner otra equivalente, y dejar que la seguridad dependiera del mismo programa que podía fallar.

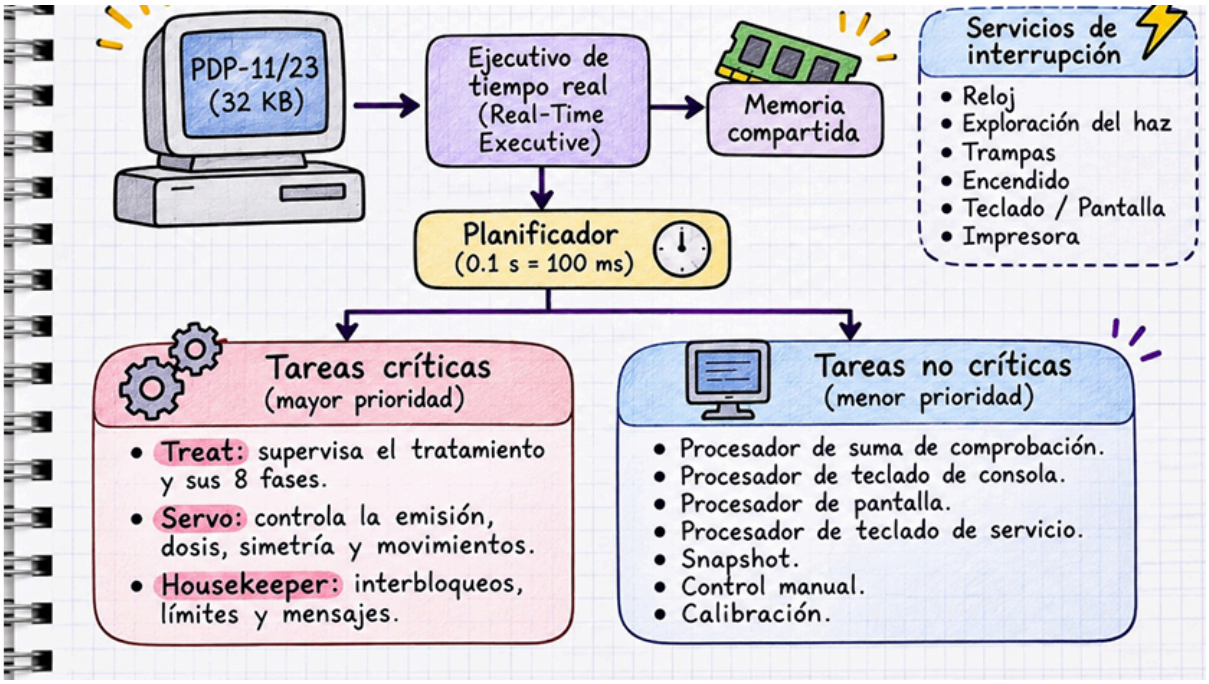
Hay un dato que lo muestra muy bien. Parte del código del Therac-25 se reutilizó del Therac-20, y el Therac-20 tenía el mismo error que causó los accidentes de Tyler. En la Universidad de Chicago, cuando los estudiantes capturaban datos muy rápido, el Therac-20 fundía fusibles y se apagaba, y el físico Frank Borger relacionó esas fallas con ese mismo error (Leveson y Turner, 1993). El error estaba en las dos máquinas. La diferencia es que en el Therac-20 había algo independiente del código que detenía el equipo, y en el Therac-25 ya no. Leveson (2017) señala que, treinta años después, reutilizar software sin volver a analizarlo en su nuevo contexto y confiar de más en él siguen apareciendo como factores en accidentes.

3. La arquitectura del software

La relación del Therac-25 con los sistemas operativos está en la forma en que organizaba sus tareas. No era un programa que corriera de principio a fin. Tenía que atender al mismo tiempo el teclado del operador, la pantalla, los motores, los imanes y las revisiones de seguridad, y todo eso lo resolvía con un pequeño sistema de tiempo real hecho a la medida. Corría en una minicomputadora PDP-11/23, estaba escrito en lenguaje ensamblador y, según Leveson y Turner (1993), lo desarrolló prácticamente una sola persona.

El software tenía cuatro partes:

- Datos almacenados, con la calibración del acelerador y las prescripciones de los pacientes.
- Un planificador apropiativo que cada 0.1 s ejecutaba primero las tareas críticas y luego, en el tiempo que sobraba, las no críticas.
- Tareas críticas, como Treat (controla las 8 fases del tratamiento con una variable llamada Tphase), Servo (haz, dosis y movimientos) y Housekeeper (interbloqueos y mensajes).
- Tareas no críticas y servicios de interrupción, entre ellas el procesador del teclado, la pantalla, el reloj y la impresora.



El detalle más importante para esta materia es cómo se comunicaban las tareas. Lo hacían con variables compartidas en memoria, sin ningún mecanismo de sincronización, y Leveson y Turner (1993) señalan que revisar el valor de esas variables y modificarlo ("test" y "set") no eran operaciones indivisibles. En clase vimos que si dos hilos pueden leer y escribir el mismo dato, y la revisión y el cambio no son atómicos ni están protegidos con un candado, el resultado depende de cómo los intercale el planificador (Wolf et al., 2015). Con un planificador apropiativo eso puede pasar en cualquier instrucción.

Con esa arquitectura, las investigaciones identificaron cuatro problemas principales. Los dos primeros son los que voy a desarrollar con código.

1. Condición de carrera en la captura de datos (Tyler, Texas, 1986).
2. Desbordamiento del contador Class3 (Yakima, Washington, 1987).
3. Mensajes de error que no explicaban nada y permitían continuar con una sola tecla.
4. Seguridad concentrada en el mismo software, sin una capa independiente que detuviera el haz.

La siguiente tabla resume los seis accidentes. No todos tuvieron la misma causa, y en los primeros tres no se pudo determinar con certeza qué ocurrió.

Fecha	Lugar	Causa identificada	Qué pasó
3 jun 1985	Marietta, Georgia	No determinada	Lesiones graves. AECL sostuvo que el equipo no podía haberlas provocado.
26 jul 1985	Hamilton, Ontario	Se sospechó de la mesa	AECL modificó los microinterruptores de la mesa, pero el problema de fondo siguió.
Dic 1985	Yakima, Washington	No determinada	Tardó en reconocerse como sobredosis. AECL negó que el equipo fuera responsable.
21 mar 1986	Tyler, Texas	Condición de carrera	Apareció "Malfunction 54" al corregir el modo. El paciente murió meses después.
11 abr 1986	Tyler, Texas	Condición de carrera	Mismo error. El físico Fritz Hager logró reproducirlo. El paciente murió.
17 ene 1987	Yakima, Washington	Desbordamiento	La revisión del colimador se saltó en la pasada 256. El paciente murió.

4. La condición de carrera de Tyler

4.1 Qué pasó

El 21 de marzo de 1986, en el East Texas Cancer Center de Tyler, la operadora capturó la prescripción de un paciente y escribió "x" (rayos X) cuando debía ser "e" (electrones). Se dio cuenta, subió con la flecha del teclado, corrigió a "e" y regresó a la línea de comandos. Lo hizo en pocos segundos porque ya tenía mucha práctica. Al iniciar el tratamiento, la máquina se detuvo y mostró "Malfunction 54" y "Treatment pause". Como era común ver errores sin consecuencias, presionó la tecla P para continuar, y el paciente recibió una segunda descarga (Leveson y Turner, 1993).

4.2 Cómo pasó dentro del programa

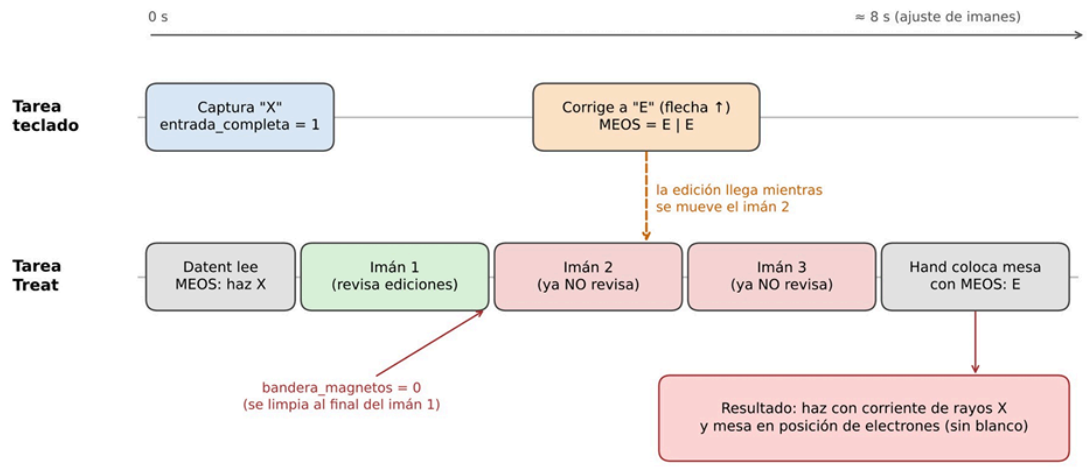
Aquí entran dos tareas que corrían al mismo tiempo, la del teclado y la de Treat. Según Leveson y Turner (1993), el funcionamiento era el siguiente:

1. La tarea del teclado guardaba el modo y la energía en una variable compartida de 2 bytes llamada MEOS. Treat, en su subrutina Datent, usaba el byte alto para buscar

en una tabla los parámetros del haz. Otra tarea, Hand, usaba el byte bajo para mover la mesa giratoria.

2. Cuando el cursor llegaba a la línea de comandos, el teclado ponía en 1 una bandera de "captura completa". Datent la veía, configuraba el haz y llamaba a Magnet para ajustar los imanes de flexión, lo cual tardaba unos 8 segundos.
3. Magnet llamaba a Ptime una vez por cada imán. Ptime sí revisaba si el operador había editado algo, pero solo si estaba encendida la bandera de imanes, y esa bandera se apagaba al terminar el primer Ptime. Cualquier corrección hecha durante el resto del ajuste ya no se detectaba.
4. Al regresar de Magnet, Datent volvía a ver la bandera de "captura completa" y pasaba a la siguiente fase sin volver a leer MEOS. Pero esa bandera solo indicaba que el cursor había llegado a la línea de comandos, no que siguiera ahí.

El resultado es que el haz quedaba configurado con los parámetros de rayos X (leídos antes de la corrección), mientras que Hand movía la mesa a la posición de electrones (con el valor ya corregido). Es decir, corriente alta sin blanco. Son dos errores de concurrencia juntos. Uno es una bandera que se limpia en el lugar equivocado, y el otro es el patrón clásico de "revisar y luego actuar" (check-then-act) sobre un dato que otra tarea puede cambiar en medio.



Condición de carrera en Tyler

4.3 Reproducción en C

Para entenderlo mejor lo simulé con dos hilos. El hilo tarea_teclado hace lo que hacía la operadora y el hilo tarea_treat hace lo de Datent, Magnet y Ptime. Escalé el tiempo para que 1 s de la máquina sea 0.1 s del programa. Las variables compartidas no tienen ninguna protección, igual que en el original. El programa recibe como argumento cuántos milisegundos espera el operador antes de corregir.

Código 1. tyler_bug.c, simulación del error

```
None
void *tarea_teclado(void *arg) {
    meos = (MODO_X << 8) | MODO_X;     /* el operador escribe "X"
por error */
    entrada_completa = true;           /* cursor llega a la línea
de comandos */
    printf("[Teclado] prescripción capturada: %s\n",
nombre(MODO_X));
    usleep(retardo_edicion_ms * 1000); /* después corrige con la
flecha arriba */
    hubo_edicion = true;
    meos = (MODO_E << 8) | MODO_E;
    printf("[Teclado] el operador corrige a %s\n",
nombre(MODO_E));
    return NULL;
}


int main(int argc, char *argv[]) {
    pthread_t t1, t2;
    if (argc > 1) retardo_edicion_ms = atoi(argv[1]);
    pthread_create(&t1, NULL, tarea_treat, NULL);
    pthread_create(&t2, NULL, tarea_teclado, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Lo ejecuté con dos tiempos distintos. Si la corrección llega pronto (200 ms, o sea 2 s de la máquina), todavía está corriendo el primer imán, Ptime la detecta y Datent vuelve a leer MEOS. Si llega un poco después (400 ms, o 4 s de la máquina), la bandera ya se apagó y se produce la combinación peligrosa.

Salida 1. Mismo programa, dos tiempos de corrección

```
None
$ gcc -Wall -pthread tyler_bug.c -o tyler_bug
$ ./tyler_bug 200
[Teclado] prescripción capturada: rayos X
[Treat] Datent configura el haz para rayos X
[Teclado] el operador corrige a electrones
```

```
[Treat] Datent configura el haz para electrones
[Treat] Haz: electrones | Mesa giratoria: electrones

$ ./tyler_bug 400
[Teclado] prescripción capturada: rayos X
[Treat] Datent configura el haz para rayos X
[Teclado] el operador corrige a electrones
[Treat] Haz: rayos X | Mesa giratoria: electrones
[Treat] !! Corriente de rayos X sin blanco: SOBREDOSIS
```

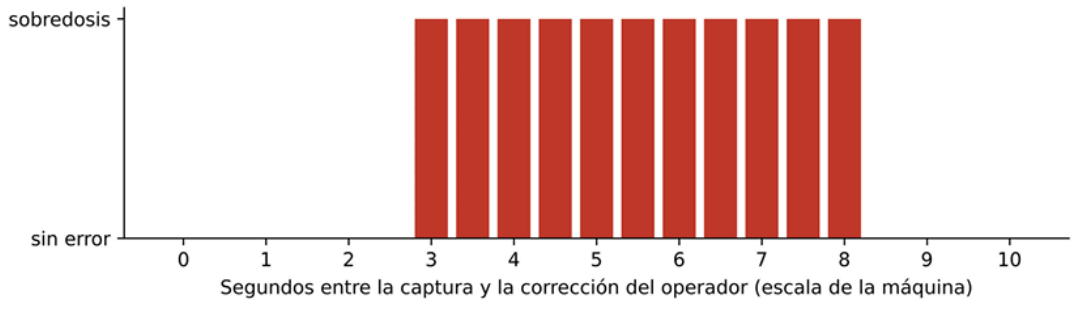
4.4 Probar los cambios rápidos

Este tipo de error es muy difícil de encontrar si solo se prueba "como debe usarse". Para ver qué tan angosta era la ventana, hice un barrido con un ciclo en bash que corre el programa con tiempos de corrección de 0 a 1000 ms, de 50 en 50, y cuenta cuántas veces aparece la sobredosis.

Código 2. Prueba de barrido de tiempos

```
None
for d in $(seq 0 50 1000); do
    echo "$d $(./tyler_bug $d | grep -c SOBREDOSIS)"
done
```

En la gráfica se ve el resultado. Solo falla si el operador corrige entre unos 3 y 8 segundos después de capturar. Antes de eso la edición se detecta, y después el tratamiento ya había comenzado. Esto ayuda a entender por qué los ingenieros de AECL no lograron reproducir el error cuando revisaron la máquina, ya que para caer en esa ventana había que capturar y corregir muy rápido. Fue el físico del hospital, Fritz Hager, quien lo reprodujo practicando con la operadora hasta escribir tan rápido como ella (Leveson y Turner, 1993).



Tiempo de corrección contra sobredosis

Además del barrido, hoy existen herramientas que detectan este tipo de problema aunque la ejecución "salga bien". Compilé la simulación con ThreadSanitizer (-fsanitize=thread), una herramienta de gcc y clang que registra cada acceso a memoria de cada hilo y avisa cuando dos hilos tocan el mismo dato sin sincronización (Serebryany e Iskhodzhanov, 2009). En la versión con error reportó dos carreras de datos, una sobre entrada_completa y otra sobre hubo_edicion.

Salida 2. ThreadSanitizer sobre la versión con error (resumida)

```
None
$ gcc -g -fsanitize=thread -pthread tyler_bug.c -o tb && ./tb 400
WARNING: ThreadSanitizer: data race (pid=1070)
SUMMARY: ThreadSanitizer: data race tyler_bug.c:55 in
tarea_teclado
WARNING: ThreadSanitizer: data race (pid=1070)
SUMMARY: ThreadSanitizer: data race tyler_bug.c:58 in
tarea_teclado
```

4.5 Corrección con los mecanismos que vimos en clase

La corrección que propongo usa tres de los mecanismos que vimos en clase (Silberschatz et al., 2018; Wolf et al., 2015). Un mutex para que la prescripción solo se lea o se modifique dentro de una sección crítica, una variable de condición para que Treat espere la captura sin estar revisando una bandera en un ciclo, y un número de versión que aumenta cada vez que el operador cambia algo. Treat guarda la versión con la que empezó y la vuelve a revisar en cada imán y justo antes de autorizar. Si cambió, reinicia la preparación.

Código 3. tyler_fix.c, versión sincronizada

```
None
/* Misma simulación con mutex, variable de condición y número de
versión */
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

enum { MODO_X = 1, MODO_E = 2 };
typedef struct { int modo; unsigned version; bool lista; }
Prescripcion;
```

```
Prescripcion presc = { 0, 0, false };           /* recurso
compartido */
pthread_mutex_t candado = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t hay_cambio = PTHREAD_COND_INITIALIZER;
int retardo_edicion_ms = 400;

const char *nombre(int m) { return m == MODO_X ? "rayos X" :
"electrones"; }


/* El teclado SOLO modifica la prescripción dentro de la sección
crítica */
void capturar(int modo) {
    pthread_mutex_lock(&candado);
    presc.modo = modo;
    presc.version++;                  /* cualquier cambio
invalida lo anterior */
    presc.lista = true;
    pthread_cond_signal(&hay_cambio);
    pthread_mutex_unlock(&candado);
}


/* ¿La versión que estoy usando sigue siendo la vigente? */
bool sigue_vigente(unsigned mi_version) {
    pthread_mutex_lock(&candado);
    bool ok = (presc.version == mi_version);
    pthread_mutex_unlock(&candado);
    return ok;
}


void *tarea_treat(void *arg) {
    Prescripcion copia;
    for (;;) {
        pthread_mutex_lock(&candado);
        while (!presc.lista)          /* espera sin consumir CPU
*/
            pthread_cond_wait(&hay_cambio, &candado);
```

```
        copia = presc;                /* foto consistente: modo +
versión */
        pthread_mutex_unlock(&candado);

        printf("[Treat] preparando %s (versión %u)\n",
               nombre(copia.modo), copia.version);
        bool interrumpido = false;
        for (int i = 0; i < 3 && !interrumpido; i++) {     /*
tres imanes */
            usleep(270000);
            interrumpido = !sigue_vigente(copia.version); /* en
CADA imán */
        }
        if (interrumpido) {
            printf("[Treat] la prescripción cambió: se
reinicia\n");
            continue;
        }

        /* Última comprobación justo antes de disparar, dentro
del candado */
        pthread_mutex_lock(&candado);
        bool ok = (presc.version == copia.version);
        int mesa = presc.modo;        /* Hand coloca la mesa con
este valor */
        pthread_mutex_unlock(&candado);
        if (!ok) {
            printf("[Treat] cambio de último momento: se
reinicia\n");
            continue;
        }

        printf("[Treat] Haz: %s | Mesa giratoria: %s ->
tratamiento autorizado\n",
               nombre(copia.modo), nombre(mesa));
        return NULL;
    }
}
```

```
void *tarea_teclado(void *arg) {
    capturar(MODO_X);
    printf("[Teclado] prescripción capturada: %s\n",
nombre(MODO_X));
    usleep(retardo_edicion_ms * 1000);
    capturar(MODO_E);
    printf("[Teclado] el operador corrige a %s\n",
nombre(MODO_E));
    return NULL;
}


int main(int argc, char *argv[]) {
    pthread_t t1, t2;
    if (argc > 1) retardo_edicion_ms = atoi(argv[1]);
    pthread_create(&t1, NULL, tarea_treat, NULL);
    pthread_create(&t2, NULL, tarea_teclado, NULL);
    pthread_join(t2, NULL);
    pthread_join(t1, NULL);
    return 0;
}
```

Salida 3. La versión sincronizada detecta la corrección

```
None
$ ./tyler_fix 400
[Teclado] prescripción capturada: rayos X
[Treat] preparando rayos X (versión 1)
[Teclado] el operador corrige a electrones
[Treat] la prescripción cambió: se reinicia
[Treat] preparando electrones (versión 2)
[Treat] Haz: electrones | Mesa giratoria: electrones ->
tratamiento autorizado
```

Con el mismo barrido de 0 a 1000 ms la versión corregida no produjo ninguna combinación peligrosa, y ThreadSanitizer no reportó ninguna carrera. Algunas cosas que quiero destacar de esta solución:

- Copiar el modo y la versión dentro del candado garantiza que Treat trabaje con una "foto" consistente, aunque el teclado cambie el dato un instante después.
- El mutex por sí solo no hubiera bastado. Si Treat leyera el modo protegido, soltara el candado y nunca volviera a revisar, el error de Tyler seguiría ocurriendo. Lo que lo evita es volver a validar la versión antes de actuar.
- Para la exclusión mutua también se podría usar un semáforo binario (sem_t inicializado en 1) en lugar del mutex. En un equipo real, además, el estado pasaría a "irradiando" dentro del mismo candado y el teclado rechazaría cualquier edición a partir de ese momento.

Lo que hizo AECL en su momento me sorprendió. La primera "solución" que envió a los hospitales fue pedir que quitaran la tecla de flecha hacia arriba del teclado y cubrieran sus contactos con cinta aislante (Leveson y Turner, 1993). Después corrigió el software de dos formas que coinciden con lo que se ve en la simulación. Movió el apagado de la bandera de imanes al final de Magnet, y agregó una variable controlada por el teclado que indica si el cursor sigue fuera de la línea de comandos, para que Treat no avance mientras la captura siga en curso.

5. El desbordamiento de Class3 en Yakima

5.1 Qué pasó

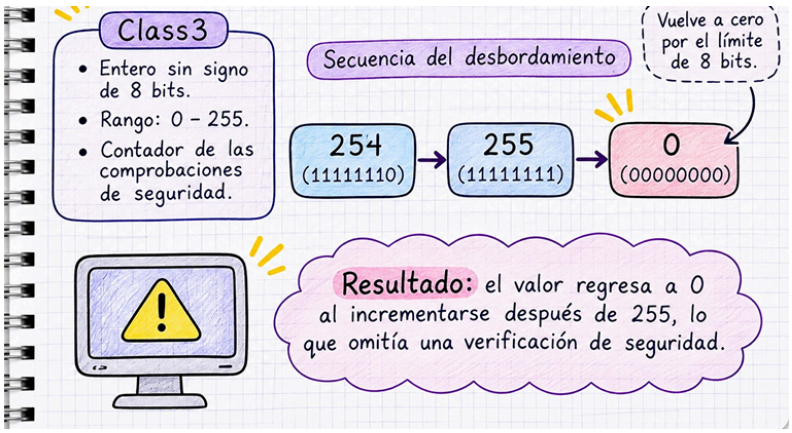
El 17 de enero de 1987, en Yakima, el operador tomó una placa de verificación con la mesa en la posición de luz de campo y después presionó el botón "set" para continuar. La máquina disparó el haz completo de electrones de 25 MeV sin blanco y sin barrido, con la mesa todavía en la posición que nunca debe usarse para irradiar (Leveson y Turner, 1993).

5.2 Cómo pasó dentro del programa

Mientras el equipo se preparaba, una rutina llamada Set-Up Test se ejecutaba cientos de veces seguidas. En cada pasada incrementaba una variable compartida de un byte llamada Class3, cuyo valor distinto de cero significaba "hay que revisar la posición del colimador superior". La tarea Housekeeper, en su subrutina Lmtchk, llamaba a Chkcol para hacer esa revisión solo si Class3 era distinto de cero (Leveson y Turner, 1993).

Un byte sin signo solo guarda valores de 0 a 255. En la pasada 256, Class3 valía 255 y al sumarle 1 regresaba a 0, así que esa vez la revisión del colimador se saltaba. Si el operador presionaba "set" justo en ese momento, la máquina no se enteraba de que la mesa estaba mal colocada. Aunque es un error aritmético, también tiene algo de concurrencia, porque depende de que coincidan una pasada concreta de Set-Up Test y la acción del operador.

Desbordamiento de Class3



5.3 Reproducción en C

Código 4. class3.c, versión original y versión corregida

```C#
/* El error de Yakima: un contador de 8 bits usado como condición
de seguridad. */
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

bool colimador_en_posicion = false;       /* la mesa sigue en
"luz de campo" */


/* Versión original: Class3 se INCREMENTA en cada pasada de
Set-Up Test */
bool setup_test_original(uint8_t *class3) {
    (*class3)++;                          /* 255 + 1 = 0 */
    if (*class3 != 0)                     /* Lmtchk llama a Chkcol
solo si != 0 */
        return colimador_en_posicion; /* Chkcol revisa la
posición real */
    return true;                          /* con 0 se salta la
revisión */
}


/* Corrección de AECL: asignar un valor fijo distinto de cero */
bool setup_test_corregido(uint8_t *class3) {
```

```
    *class3 = 1;                        /* nunca puede llegar a 0
*/
    return colimador_en_posicion;       /* la revisión se hace
siempre */
}


int main(void) {
    uint8_t c_orig = 0, c_corr = 0;
    for (int pasada = 1; pasada <= 300; pasada++) {
        bool ok_orig = setup_test_original(&c_orig);
        bool ok_corr = setup_test_corregido(&c_corr);
        if (ok_orig)
            printf("Pasada %d: Class3 = %u -> original
AUTORIZA\n",
                   pasada, c_orig);
        if (ok_corr)
            printf("Pasada %d: corregida autoriza\n", pasada);
    }
    printf("Fin: la versión corregida nunca autorizó con la mesa
mal colocada\n");
    return 0;
}
```

Salida 4. En 300 pasadas, la original autoriza una vez

```
None
$ gcc -Wall -Wextra class3.c -o class3 && ./class3
Pasada 256: Class3 = 0 -> original AUTORIZA
Fin: la versión corregida nunca autorizó con la mesa mal colocada
```

La corrección de AECL fue muy simple. En lugar de incrementar Class3, Set-Up Test le asigna siempre un valor fijo distinto de cero (Leveson y Turner, 1993), que es lo que hace setup_test_corregido(). Hay dos cosas de este error que me parecen importantes:

- Compilé con -Wall -Wextra y gcc no dio ninguna advertencia. En C el desbordamiento de un entero sin signo no es un error, está definido que "da la vuelta" a cero. Por eso no basta con que el programa compile limpio, y este tipo de error solo se encuentra con pruebas de límites (probar 254, 255 y 256) o con analizadores estáticos.

- El fondo del problema no es el tamaño del tipo. Usar un contador como si fuera un valor booleano ("¿hay que revisar?") es lo que permite que un valor aritmético se convierta en una autorización. Una condición de seguridad debe representarse explícitamente, y si por alguna razón no se puede evaluar, el sistema debe asumir lo peor.

6. Mensajes de error y la tecla P

El tercer problema no es de concurrencia, pero agravó todo lo anterior. "Malfunction 54" significaba "dose input 2", y el manual del operador no lo explicaba. La máquina lo mostraba como una pausa del tratamiento, y el operador podía reanudarlo con la tecla P hasta cinco veces. Como los operadores estaban acostumbrados a ver muchos mensajes de error que no tenían consecuencias, la reacción natural era presionar P (Leveson y Turner, 1993). Dentro del plan de correcciones de 1987 se cambió ese comportamiento para que las fallas relacionadas con la dosis suspendieran el tratamiento en lugar de pausarlo, de modo que ya no pudiera reanudarse sin volver a capturar los datos.

7. ¿Cómo se resuelve esto hoy?

Pasaron casi 40 años desde estos accidentes y hoy hay software en prácticamente todo, desde equipos médicos hasta máquinas industriales y teléfonos. En esta sección intento responder cómo se evitan ahora las consecuencias de errores "pequeños" como los del Therac-25.

7.1 Un error interno es una excepción, y hay que decidir a dónde lleva

Un error interno, como una lectura imposible de un sensor o un valor fuera de rango, es una condición de falla. En lenguajes como C++, Java o Python se maneja con excepciones (try/catch), y en C con códigos de retorno. Pero el mecanismo es lo de menos. La decisión importante es en qué estado queda el sistema después del error. En un equipo peligroso la regla es diseñar "a prueba de fallas" (fail-safe), es decir, que ante cualquier error, incluso uno que nadie previó, el equipo vaya al estado seguro, que en este caso es no irradiar.

El siguiente programa aplica esa idea. Valida la prescripción en el momento en que entra, concentra en una sola función la decisión de qué hacer con cada error y, en el default del switch, cualquier error desconocido detiene el equipo. Además, a diferencia de la tecla P, un tratamiento suspendido no se puede reanudar sin volver a capturar y verificar todo.

Código 5. errores.c, manejo de errores a prueba de fallas

None

```
/* Manejo de errores "a prueba de fallas": ante la duda, el
sistema se detiene. */
#include <stdio.h>
#include <stdbool.h>

typedef enum { REPOSO, PREPARANDO, LISTO, IRRADIANDO, SUSPENDIDO
} Estado;
typedef enum { OK, ERR_RANGO, ERR_MESA, ERR_DOSIMETRO,
ERR_DESCONOCIDO } Error;

typedef struct { int modo; int energia_mev; int dosis_cgy; }
Prescripcion;

Estado estado = REPOSO;

Error validar(const Prescripcion *p) {
    if (p->modo != 1 && p->modo != 2)         return ERR_RANGO;
    if (p->energia_mev < 5 || p->energia_mev > 25) return
ERR_RANGO;
    if (p->dosis_cgy <= 0 || p->dosis_cgy > 600)   return
ERR_RANGO;
    return OK;
}


/* Un solo lugar decide qué pasa con cada error. Lo que no se
reconoce, detiene. */
void manejar(Error e) {
    switch (e) {
    case OK:
        return;
    case ERR_RANGO:                 /* dato inválido: se rechaza
y se explica */
        printf("Prescripción fuera de rango: corrija el dato\n");
        estado = REPOSO;
        break;
    case ERR_MESA:
    case ERR_DOSIMETRO:             /* falla de seguridad: sin
"continuar" */
```

```
        printf("Falla de seguridad (%s): tratamiento
SUSPENDIDO,\n "
               "requiere reinicio y nueva captura\n",
               e == ERR_MESA ? "posición de la mesa" :
"dosímetro");
        estado = SUSPENDIDO;
        break;
    default:                         /* error que nadie previó:
también detiene */
        printf("Error desconocido %d: SUSPENDIDO\n", e);
        estado = SUSPENDIDO;
    }
}


bool reanudar(void) {               /* la "P" del Therac-25 ya no
existe */
    if (estado == SUSPENDIDO) {
        printf("No se puede reanudar: hay que capturar y
verificar todo\n");
        return false;
    }
    return true;
}


int main(void) {
    Prescripcion buena = { 2, 10, 180 }, mala = { 2, 40, 180 };
    manejar(validar(&mala));
    manejar(validar(&buena));
    estado = IRRADIANDO;
    manejar(ERR_DOSIMETRO);         /* el equivalente a
"Malfunction 54" */
    reanudar();
    return 0;
}
```

Salida 5. Un dato inválido se rechaza y una falla de dosimetría no deja continuar

```
None
$ ./errores
Prescripción fuera de rango: corrija el dato
Falla de seguridad (dosímetro): tratamiento SUSPENDIDO,
  requiere reinicio y nueva captura
No se puede reanudar: hay que capturar y verificar todo
```

Comparado con el Therac-25, aquí el error dice qué pasó, distingue un dato mal capturado de una falla de seguridad, y no deja que la costumbre del operador decida si se continúa.

7.2 Probar las situaciones poco comunes

Los dos errores principales del Therac-25 tienen algo en común, pues solo aparecían en casos raros, como una corrección muy rápida o la pasada número 256. Hoy esos casos se buscan a propósito con varias técnicas.

- Pruebas de límites, que prueban justo los valores donde cambia el comportamiento (0, 255, 256, energía mínima y máxima).
- Pruebas de temporización y estrés, como el barrido de la sección 4.4, que repiten una operación con distintos tiempos y órdenes de ejecución.
- Detectores de carreras como ThreadSanitizer, que encuentran accesos sin sincronizar aunque el programa "funcione".
- Análisis estático y revisión de código por otra persona, algo que en el Therac-25 prácticamente no existió porque el software lo hizo y lo revisó una sola persona.

7.3 ¿Protecciones físicas o protecciones de software?

Una conclusión fácil del caso sería decir "hay que regresar a los interbloqueos físicos". Pero eso va en contra de cómo ha evolucionado la tecnología, donde cada vez más funciones que antes eran de hardware se hacen por software. Un buen ejemplo es el teléfono celular. Tiene un módem programable que funciona como un radio de amplio espectro con potencia suficiente para dañar tejido si se usara mal, y las protecciones que lo evitan son de software. El firmware del módem limita la potencia máxima, y muchos teléfonos la reducen todavía más cuando un sensor de proximidad detecta que el equipo está pegado a la cabeza.

Pensándolo con el ejemplo del Therac-20, el fusible no protegía por ser hardware, sino porque era independiente del código que tenía el error. Esa independencia hoy se puede lograr con software, y varias de las formas de hacerlo son temas de esta materia:

- Procesadores separados. El módem del celular corre en su propio procesador con su propio firmware, y una aplicación no puede ordenarle directamente que transmita con más potencia. Tiene que pedírselo al sistema operativo, que a su vez pasa por el controlador y el firmware que aplican los límites.

- Separación de privilegios. El modo kernel y el modo usuario, junto con la protección de memoria (MMU), impiden que un error en una aplicación sobrescriba la memoria del núcleo o de otra tarea. En el Therac-25 cualquier tarea podía escribir cualquier variable compartida.
- Watchdog. Es un temporizador que reinicia o detiene el sistema si el software deja de "reportarse" a tiempo. Sirve para detectar que el programa se colgó o perdió el control.
- Particiones. En aviónica, por ejemplo, un hipervisor separa en memoria y en tiempo de procesador las tareas de distinta criticidad para que una no pueda afectar a otra.
- Verificación formal. El microkernel seL4 tiene una demostración matemática de que su implementación cumple su especificación (Klein et al., 2009), así que la capa que hace cumplir el aislamiento es mucho más confiable que el resto del sistema.

Con esto entiendo que la lección del Therac-25 no es "hardware sí, software no". La función de seguridad debe ser más simple que lo que protege, debe estar aislada para que no comparta los mismos errores, y debe poder verificarse por separado. En el Therac-25 la seguridad era un pedazo más del mismo programa, con las mismas variables compartidas y los mismos errores.

7.4 Cómo lo hacen los aceleradores actuales

Los aceleradores médicos de hoy también están controlados por computadora, pero con varias capas independientes. Por ejemplo, la norma particular para aceleradores de electrones, IEC 60601-2-1 (International Electrotechnical Commission [IEC], 2020), y también las regulaciones de radioprotección de varios estados de Estados Unidos, piden dos sistemas de monitoreo de dosis, cada uno capaz de terminar la irradiación por su cuenta, diseñados de manera que la falla de uno no afecte al otro. Además, la posición de los componentes se confirma con más de un sensor, algo que AECL también agregó al Therac-25 en 1987 con un potenciómetro en la mesa giratoria (Leveson y Turner, 1993).

El software en sí también se desarrolla con reglas distintas. La norma IEC 62304 (IEC, 2015) clasifica el software médico según el daño que podría causar, y al software que podría provocar lesiones graves o la muerte le exige más documentación, pruebas y control de cambios. La norma ISO 14971 (International Organization for Standardization [ISO], 2019) obliga a analizar desde el diseño qué puede salir mal y qué medida controla cada riesgo. En el Therac-25, según Leveson y Turner (1993), el análisis de seguridad que hizo AECL prácticamente no consideró el software.

8. ¿Y la inteligencia artificial?

Al principio pensé en comparar el caso con la IA que interpreta radiografías, pero creo que ahí no cambia mucho. Desde siempre, cuando alguien recibe un estudio, la lectura del técnico es solo una primera información y quien diagnostica es el médico tratante, y lo mismo pasa con la lectura que dé un sistema de IA. Donde la lección del Therac-25 sí aplica directamente es en el software que actúa sin que una persona revise cada decisión, como las bombas de insulina de lazo cerrado que ajustan la dosis solas. Ahí vuelven a ser

importantes la validación de datos, el manejo de errores hacia un estado seguro y las protecciones independientes, sin importar si la decisión la toma un algoritmo clásico o un modelo de IA.

9. Reflexión

Lo que más me llama la atención de este caso es que no se trató simplemente de una máquina que presentó una falla. Fueron varios problemas que se fueron acumulando y que, en diferentes momentos, pudieron haberse investigado y corregido antes de que más pacientes resultaran afectados. Antes de este trabajo, para mí un error de programación en el peor de los casos hacía que el programa tronara o diera un resultado incorrecto. Aquí, errores que en una práctica de la escuela no tendrían mayor consecuencia, como una bandera mal puesta o un contador de un byte, terminaron lastimando a personas.

También me sorprendió lo difícil que fue encontrar el error. La máquina funcionó bien con cientos de pacientes y el problema solo aparecía si alguien escribía rápido en una ventana de pocos segundos. Al simularlo yo misma y ver el mensaje de sobredosis aparecer solo al cambiar el retardo de 200 a 400 ms, me quedó mucho más claro que leyéndolo, y entendí mejor por qué en clase insistimos tanto en proteger las secciones críticas aunque "parezca que funciona".

Muchas veces pensamos que, por el hecho de que una máquina esté controlada por una computadora, necesariamente será más precisa y segura. Al principio yo creía que el error había sido confiar en el software, pero al revisar el caso con más detalle me di cuenta de que el problema no fue usar software, sino no comprender completamente cómo podía fallar y no tener nada independiente que lo detuviera cuando falló. Los pacientes no tienen por qué conocer los problemas de programación, los errores de diseño o las limitaciones técnicas de una máquina. Ellos simplemente confían en que el equipo que utiliza el hospital es seguro.

Por eso considero que, como futuros ingenieros, debemos entender que nuestro trabajo tiene consecuencias más allá de lo técnico. El Therac-25 me deja una enseñanza muy clara. Un error puede parecer pequeño mientras se encuentra dentro de una computadora, pero sus consecuencias pueden ser enormes cuando ese sistema interviene directamente en la vida de alguien.

10. Conclusión

Los accidentes del Therac-25 que se pudieron explicar tuvieron causas muy concretas. En Tyler, dos tareas compartían variables sin sincronización, y una corrección hecha entre 3 y 8 segundos después de la captura no se detectaba, lo que dejaba el haz configurado para rayos X con la mesa en posición de electrones. En Yakima, un contador de 8 bits usado como condición de seguridad regresaba a cero en la pasada 256 y se saltaba la revisión del colimador. Al reproducir ambos errores en C, se ve que se resuelven con herramientas que

ya conocemos, como un mutex con revalidación antes de actuar, una variable de condición en lugar de una bandera, y una condición explícita en lugar de un contador.

La otra conclusión es que el problema no fue usar software, sino que la seguridad dependía del mismo programa que fallaba. Hoy los equipos son todavía más dependientes del software, y lo que los hace seguros es que las funciones de seguridad están aisladas, son más simples y se pueden verificar por separado. Para lograrlo se usan procesadores separados, protección de memoria, watchdogs y núcleos verificados, además de manejar los errores hacia un estado seguro y probar a propósito los casos raros.

Referencias

International Electrotechnical Commission. (2015). IEC 62304:2006+AMD1:2015. Medical device software. Software life cycle processes. IEC.

International Electrotechnical Commission. (2020). IEC 60601-2-1:2020. Medical electrical equipment. Part 2-1: Particular requirements for the basic safety and essential performance of electron accelerators in the range 1 MeV to 50 MeV. IEC.

International Organization for Standardization. (2019). ISO 14971:2019. Medical devices. Application of risk management to medical devices. ISO.

https://www.iso.org/standard/72704.html

Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H. y Winwood, S. (2009). seL4: Formal verification of an OS kernel. En Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (pp. 207–220). ACM.

https://doi.org/10.1145/1629575.1629596

Leveson, N. G. (2017). The Therac-25: 30 years later. Computer, 50(11), 8–11.

https://doi.org/10.1109/MC.2017.4041349

Leveson, N. G. y Turner, C. S. (1993). An investigation of the Therac-25 accidents. Computer, 26(7), 18–41.

https://doi.org/10.1109/MC.1993.274940

Serebryany, K. e Iskhodzhanov, T. (2009). ThreadSanitizer: Data race detection in practice. En Proceedings of the Workshop on Binary Instrumentation and Applications (pp. 62–71). ACM.

https://doi.org/10.1145/1791194.1791203

Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). Operating system concepts (10.ª ed.). Wiley.

Wolf, G., Ruiz, E., Bergero, F. y Meza, E. (2015). Fundamentos de sistemas operativos. Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas y Facultad de Ingeniería.